Less Data, Smaller Model? Data-Constrained Training of Small GPTs under a Fixed Compute Budget

Timothy Teo Shao Jun (A0272851B)
Delphino Chew Jing Cheng (A0271802L)

Repository: `https://github.com/Teamoreteateo/DSA4212-Small_LLM`

DSA4212, 13 November 2026

Declarations (required by the project specification).

- **Compute budget:** a fixed number of training tokens per model, equal to 600 s of median measured training throughput on one NVIDIA Tesla T4 (Section 3); evaluation, tokenizer training and pilot runs are excluded. Declared on 27 Oct 2026, before the main experiments.
- **Initialisation:** all models are trained from random initialisation; no pretrained weights, embeddings or other neural components are used. The tokenizer is trained by us on the training split.
- **Evaluation objective:** bits per byte (BPB) on a held-out test set, the second half of the official TinyStories validation split, which is never used for tuning.

1 Introduction

Compute-optimal scaling laws [**??**] tell us how to split a training budget between model size and training tokens, assuming fresh data is always available. When unique data runs short, extra compute must go either into more epochs over the same data or into a larger model that sees it fewer times. **?** studied this trade-off for models of 10M–9B parameters: up to about four epochs of repetition was almost as good as new data, and extra compute was best spent mostly on more epochs rather than more parameters. We ask whether this holds for tiny GPTs trained for ten minutes on one GPU:

> *Under a fixed per-run compute budget, how do (i) the optimal model size and (ii) the usefulness of repeating data change as the number of unique training tokens U shrinks?*

We fixed two hypotheses before the main experiments. **H1**: the optimal model size decreases as U decreases. **H2**: larger models stop benefiting from repetition after fewer epochs. Across five model sizes (0.4M–25M non-embedding parameters), four data levels (1.5M–100M tokens), three seeds and a dropout follow-up, H2 is supported. H1 holds only in part: without regularisation the optimum moves non-monotonically, because larger models are slower and so repeat their data less; with dropout it is unchanged at 6M tokens and shifts to the smallest model only at 1.5M.

2 Background

Compute-optimal scaling. **?** found power laws linking loss to parameters, data and compute, and counted non-embedding parameters, a convention we follow. **?** showed that parameters and training tokens should grow in equal proportion, about 20 tokens per parameter in their Table 3 (their counts include embeddings). Their analysis runs used less than one epoch of data.

Repeated data. **?** extend this to repeated data, treating each repetition as worth exponentially less than a fresh token. Their fitted half-life is about 16 epochs and repetition is worthless by about 40; because excess parameters lose value faster than repeated data, they recommend smaller models trained for more epochs (basis of H1). They also note that too many epochs eventually *increase* loss, which their law does not capture (basis of H2). **?** show that repeating a small subset many times is especially harmful; like Muennighoff et al., we repeat the whole subset uniformly. **?** show that test error can be non-monotonic in model size and training time (double descent).

Our setting. TinyStories [**?**] was built so that models below 10M parameters can write fluent English, which suits ten-minute runs; our code follows nanoGPT [**?**]. Our models (0.4M–25M parameters) lie at and below the bottom of Muennighoff et al.'s range (10M–9B), the data is simple synthetic text, and the budget is matched in device time rather than FLOPs.

3 Experimental setup

Our design follows the data-constrained protocol of **?** at a much smaller scale, with one deliberate difference: the per-run budget is matched in *device time* rather than in FLOPs (Section 3).

Data. We use TinyStories [**?**], whose simple vocabulary allows models with only a few million parameters to learn fluent text, making it suitable for ten-minute runs. Training stories are shuffled once with a fixed seed and tokenised; the first 100M tokens form the training pool. Each unique-data level $U \in \{100, 25, 6, 1.5\}$M uses the first U tokens of this pool, so smaller subsets are nested inside larger ones, as in **?**, Fig. 2: differences between levels reflect the *amount* of unique data, not its content. The official TinyStories validation split (21,990 stories) is halved: the first half (2.33M tokens) is our validation set, used only for learning-rate selection and training curves; the second half (2.56M tokens) is the test set, used only for reported numbers.

Repetition. A run cuts its U tokens into non-overlapping 256-token windows and reshuffles them every epoch, so the entire subset is repeated uniformly and epoch counts are exact. This matches **?** and avoids the partial up-sampling studied by **?**, where repeating a small fraction of data many times is especially harmful.

Tokenizer. A byte-level BPE tokenizer [**?**] with a 4096-token vocabulary is trained on 200k training stories (tokenizer training is excluded from the budget, as permitted). A small vocabulary keeps the embedding table from dominating tiny models: with GPT-2's 50k vocabulary, m1's embedding would hold ≈6.4M parameters, 16× its 0.40M non-embedding parameters; with 4096 tokens it holds 0.52M. The tokenizer averages 3.92 bytes per token on the training data and 3.96 on the validation data.

Models. Five pre-LayerNorm GPT models [**??**] with tied input/output embeddings, GELU MLPs of width $4d$, learned positional embeddings, context length 256, and GPT-2 initialisation, following nanoGPT [**?**] (Table 1). Dropout is 0 except in the follow-up (Section 4.4). We report non-embedding parameter counts, following **?**.

Optimisation. AdamW [**?**] with $\beta = (0.9, 0.95)$, weight decay 0.1 on weight matrices only, gradient clipping at 1.0, batch $64 \times 256 = 16{,}384$ tokens, and fp16 mixed precision (the T4 has no bf16). The learning rate warms up linearly over the first 2% of the budget and then follows a cosine decay to 10% of its peak, the schedule used by **?**. Because the schedule is defined on the fraction of budget used, every run completes it. Peak learning rates were selected per model in 17 pilot runs at $U = 100$M on

the *validation* set (`results/tables/lr_sweep.csv`): 3×10^{-3} for m1–m4 and 2×10^{-3} for m5, which diverged at 3×10^{-3}. A single shared learning rate was rejected because it would favour some model sizes over others and bias the comparison of optimal size.

Compute budget. Each run receives a fixed number of training tokens, equal to 600 s of that model's median measured training throughput on an NVIDIA Tesla T4 (Table 1). We first timed runs directly, but throughput for the *same* model varied by up to 24–39% across cloud hosts, which would have made results depend on host assignment (Section 5). Fixing tokens removes this noise while keeping the budget equal to ten minutes of typical T4 time. The budget was declared before the main experiments (repository commit of 27 Oct 2026).

Matching device time rather than FLOPs is a deliberate choice: it reflects the cost a practitioner pays for GPU time. Small models use the GPU less efficiently, so non-embedding training FLOPs ($6ND$) range from 1.0×10^{15} for m1 to 7.0×10^{15} for m5. Under a FLOP-matched budget, as in **?** and **?**, small models would receive even more tokens.

Table 1: Models, per-run budgets, and epochs over the unique data at each U (token budget/U). Throughput: median over pilot runs on a T4. Params and tokens/param are non-embedding.

						epochs at $U =$			
Model	layers/d/heads	params	tokens/s	budget	tokens/param	100M	25M	6M	1.5M
m1	2/128/4	0.40M	716k	430M	1083	4.3	17	72	287
m2	4/192/4	1.78M	379k	228M	128	2.3	9.1	38	152
m3	6/256/8	4.74M	234k	141M	30	1.4	5.6	24	94
m4	6/384/6	10.6M	145k	87M	8.2	0.9	3.5	15	58
m5	8/512/8	25.2M	76k	46M	1.8	0.5	1.8	7.7	31

Experimental grid. We train all 5 models at all 4 values of U with 3 seeds (60 runs). Epochs are not chosen directly: they equal token budget/U (Table 1). The U levels were chosen so that the grid spans the regimes identified by **?**: under 4 epochs (repetition nearly free), around 16 epochs (their estimated half-life of repeated data), and far beyond it, where repetition is nearly worthless and overfitting is expected.

Evaluation. The primary metric is test bits per byte, BPB $= \sum_t -\ln p(x_t \mid x_{<t}) \,/\, (\ln 2 \cdot B)$, where B is the number of UTF-8 bytes of the predicted tokens, computed on the first 2M test tokens at the final checkpoint. BPB does not depend on the tokenizer, so results can be compared with other work. Following **?**, we use no early stopping, so that overfitting remains visible. Training and validation loss are also logged every 30 s of training time on fixed windows (identical across runs) for the overfitting analysis. BPB measures next-token prediction only, not the quality of generated stories or performance outside TinyStories. We report means over 3 seeds with 95% t-intervals. Decision rules for both hypotheses, and a rule for diverged runs, were fixed before the main grid (`notes/analysis_plan.md`). No run met the divergence rule (final validation loss more than 0.5 nats above its cell's median).

4 Results

4.1 Does the optimal model size shrink with less unique data? (H1)

Data-rich regime. At $U = 100$M, m4 (0.604 ± 0.001 BPB) and m3 (0.605 ± 0.003) are tied for the lowest test BPB (Fig. 1, left; Table 2). They see 8 and 30 tokens per non-embedding parameter,

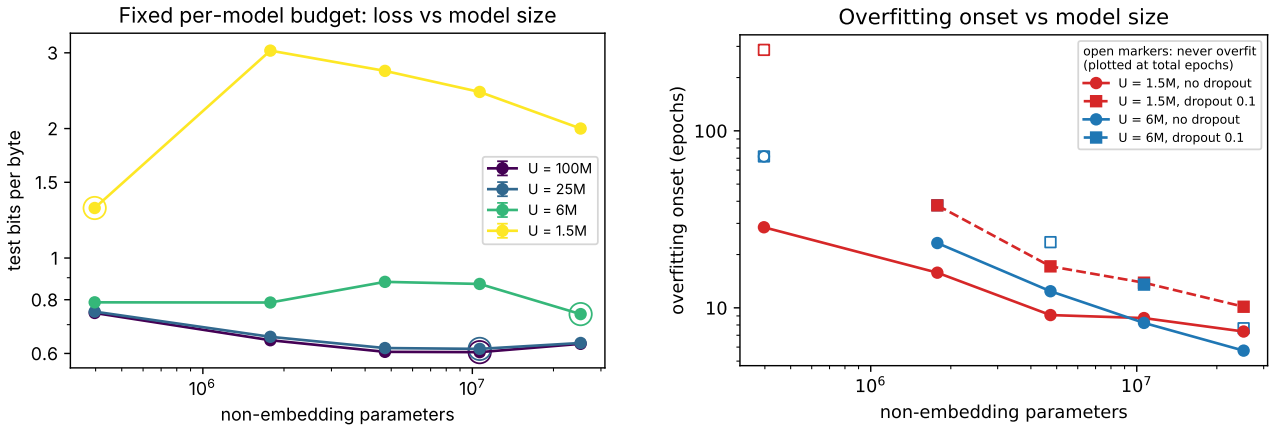


Figure 1: Left: test BPB against model size at each U (log axes; circle marks the optimum; error bars: 95% t-intervals over 3 seeds). Right: overfitting onset (epochs at minimum validation loss) against model size at 6M and 1.5M, without and with dropout; open markers are runs that never overfit, plotted at their total epochs.

bracketing the ≈20 tokens per parameter that **?** find compute-optimal.[1] Both the smallest model (m1, 1083 tokens/param) and the largest (m5, 1.8 tokens/param) are clearly worse, so the optimum lies inside our model range. The ranking at $U = 25$M is the same (m4 0.615, m3 0.618).

Repetition up to a few epochs is nearly free. Going from $U = 100$M to 25M means every model repeats its data about four times more often. Test BPB rises by only 0.5% for m5 (1.8 epochs), 0.7% for m1 (17 epochs; it already repeats its data 4.3 times at 100M) and 1.8–2.0% for m2–m4 (3.5–9 epochs). These differences exceed the confidence intervals but are small in practice, consistent with the finding of **?** that up to about four epochs of repeated data is worth almost as much as fresh data.

Scarce data: a non-monotonic path. At $U = 6$M the optimum moves *up* to the largest model, m5 (0.741 ± 0.005), ahead of m1 and m2 (≈0.79), while the mid-sized m3 and m4 are worst (0.88 and 0.87). At $U = 1.5$M it collapses to the smallest model, m1 (1.307 ± 0.015), with m2 the worst at 3.04 BPB, close to the ≈3.06 BPB of a uniform guess over the vocabulary ($\log_2 4096/3.92$). By our pre-registered rule H1 is supported: the optimum at 1.5M (m1) is smaller than the optimum at 100M (m4), and at 1.5M their intervals are far apart (1.307 ± 0.015 vs. 2.433 ± 0.019). The 6M row, however, shows that the optimum does not shrink smoothly, and Section 4.3 shows that the result depends on training to the end of the budget.

Table 2: Test BPB (mean over 3 seeds; 95% interval half-width ≤ 0.036 in every cell). Bold: optimum per row.

U	m1	m2	m3	m4	m5
100M	0.745	0.644	0.605	**0.604**	0.632
25M	0.751	0.656	0.618	**0.615**	0.635
6M	0.789	0.788	0.880	0.870	**0.741**
1.5M	**1.307**	3.036	2.723	2.433	2.000

[1]The bracket also holds if embedding parameters are counted: m3 then has 5.9M parameters (24 tokens/param) and m4 12.3M (7.1 tokens/param).

4.2 When does repetition stop helping? (H2)

We measure overfitting onset as the number of epochs at which validation loss reaches its minimum, counted only when it later rises by more than 0.005 nats. At $U = 6$M, onset falls steadily with model size: 23.2, 12.4, 8.2 and 5.7 epochs for m2–m5, while m1 never overfits in 72 epochs. At $U = 1.5$M it is 28.5, 15.8, 9.1, 8.8 and 7.3 epochs for m1–m5, after which validation loss rises by 0.9 (m1) to 5.7 (m2) nats. H2 is therefore supported (Fig. 1, right): larger models stop benefiting from repetition sooner. Because losses are logged every 30 s of training, onset is resolved only to about 1/20 of a run's epochs. This is fine at 6M (0.4–1.9 epochs), but at 1.5M m3, m4 and m5 cannot be told apart. The secondary measure, the final gap between validation and training loss, is not monotonic in size at 1.5M (m2 has the largest gap) because it depends on how many epochs a model trains *past* its onset, not only on when overfitting starts.

4.3 Mechanism: size and repetition are coupled

Under a fixed device-time budget a larger model is slower, so it sees fewer epochs of the same data. Combining this with H2 explains the H1 pattern. At 6M, m3 and m4 run 24 and 15 epochs, well past their onsets (12 and 8), and their validation loss rises by 0.40 nats; m5 runs only 7.7 epochs, just past its onset (5.7), and loses 0.08. Most of m5's advantage comes from this protection: under early stopping (below) its lead over m4 shrinks from 0.13 to about 0.01 BPB, although it remains first. m1 and m2 have too little capacity to memorise much. The non-monotonic shape resembles model-wise double descent [**?**], but here size and epochs change together, so the two effects cannot be separated. At 1.5M every model passes its onset by a wide margin, and only m1's tiny capacity limits the damage.

Sensitivity check: early stopping. As pre-registered, we re-rank models by their best validation loss (an early-stopped checkpoint) instead of the final one (`results/tables/early_stop_check.csv`). The ranking is unchanged at 100M and 25M, but at the small U levels it reverses: at 6M the order becomes m5, m4, m3, m2, m1, and at 1.5M m4 and m5 are best and m1 is *last*. Without regularisation, the shrinking optimum in H1 therefore depends on training every model to the end of its budget: early stopping alone favours larger models at small U. Section 4.4 shows that this changes once overfitting is controlled by dropout. This check ranks seed-mean validation losses without intervals, and selecting the best checkpoint on the validation set is slightly optimistic, so it supports a qualitative conclusion only.

4.4 Follow-up: does regularisation change the optimum?

If over-repetition drives the H1 pattern, regularisation should change it. We reran all five models at $U \in \{6, 1.5\}$M with dropout 0.1 [**?**], keeping budgets and learning rates fixed (30 runs; Table 3). The pre-registered follow-up covered m3 and m5; m4 and then m1, m2 were added to complete the comparison (commit history).

Moderate scarcity (6M): the reversal disappears. Dropout improves m2–m5 by 0.06–0.19 BPB and removes overfitting almost entirely (validation loss rises by at most 0.006 nats). The optimum returns to m4 (0.681 ± 0.002), clearly ahead of m5 (0.685 ± 0.001), the same model that is optimal at 100M and 25M. Once overfitting is controlled, moderate data scarcity does not move the optimum. Dropout also outperforms early stopping here: the best early-stopped model without dropout (m5) reaches $\approx$0.72 BPB, against 0.681 for m4 with dropout. Dropout slightly *hurts* m1 ($0.789 \rightarrow 0.802$): the smallest model was not overfitting, so regularisation only removes capacity it needs.

Extreme scarcity (1.5M): the smallest model wins, and wins by more. Dropout lowers test BPB for every model, by 33% (m1) to 70% (m2), and the ranking becomes almost monotonic in size: m1 (0.874 ± 0.004) < m2 (0.921) < m3 (1.151) < m5 (1.290) < m4 (1.355). m1 is best both at the final checkpoint and under early stopping (on validation, by only 0.008 BPB over m3 and m2), unlike without dropout, where early stopping put m1 last. m1 with dropout also beats the best early-stopped model without dropout (m4, ≈0.94 BPB, converted from validation loss at 3.96 bytes per token; validation BPB runs about 0.01 above test BPB). With regularisation, extreme data scarcity therefore does favour the smallest model, which supports H1 more strongly than the unregularised grid.

H2 under dropout. Dropout delays overfitting but does not prevent it at 1.5M: onset falls with size (37.9, 17.2, 13.9 and 10.1 epochs for m2–m5; m1 never overfits) and validation loss still rises by 0.1–1.3 nats afterwards. H2 thus holds with and without regularisation.

Table 3: Test BPB without and with dropout 0.1 (mean over 3 seeds; 95% half-width ≤ 0.036). Bold: best per column.

	$U = 6$M		$U = 1.5$M	
Model	none	0.1	none	0.1
m1	0.789	0.802	**1.307**	**0.874**
m2	0.788	0.713	3.036	0.921
m3	0.880	0.687	2.723	1.151
m4	0.870	**0.681**	2.433	1.355
m5	**0.741**	0.685	2.000	1.290

5 What did not work, and limitations

What did not work. (i) *Wall-clock budgets*: throughput of the same model varied by up to 24% (m5) and 39% (m4) across Modal T4 hosts; one m4 pilot run on a slow host did about 30% fewer steps and looked worse than both neighbouring learning rates. This noise rivalled the effects we measure (m3 and m4 differ by 0.001 BPB at 100M), so we switched to fixed per-model token budgets. (ii) *Instability*: m5 diverged at learning rate 3×10^{-3} (final validation loss 3.45 vs. 1.76 at 2×10^{-3}). (iii) The first LR pilot put the best rate at the grid edge for m1 and m3, so a second pilot was needed. (iv) The m3–m5 dropout runs were first run on a second team member's Modal workspace and later re-executed; results changed slightly (e.g. m5 at 1.5M: 1.288 vs. 1.290 BPB), within the 95% intervals, consistent with GPU non-determinism; we report the later runs.

Limitations. We use one simple dataset, and models at and below the bottom of **?**'s size range. The device-time budget depends on the hardware's relative throughput; a FLOP-matched budget would give small models more tokens and could move the optimum. Learning rates were tuned at $U = 100$M without dropout and reused everywhere; only one dropout rate was tried, and not at 100M or 25M. Overfitting onset is resolved to about 1/20 of a run, the early-stopping check has no intervals, and we use 3 seeds per cell. Finally, our main metric uses the final checkpoint, which drives the unregularised H1 result at small U (Section 4.3).

6 Use of AI tools

We used Claude (Anthropic), in chat and through Claude Code, to draft the training, analysis and cloud-runner code; to suggest checks (a token-budget smoke test, the early-stopping sensitivity analysis

and the bytes-per-token conversion); to draft our reading notes and check them against the original papers; and to draft most of the report text from our configuration and result files. We reviewed, edited and partly rewrote the text, reviewed and tested all code, ran all experiments ourselves, and verified every reported number against the result tables in the repository. The research question, the decision to replace wall-clock timing with a token budget, the per-model learning rates, and the extension of the dropout follow-up to all five models were our decisions.

7 Conclusion

Larger models stop benefiting from repeated data after fewer epochs than smaller models, with or without dropout (H2). Whether scarce data calls for a smaller model (H1) depends on how overfitting is handled: without regularisation, the optimum moves non-monotonically because larger models are slower and repeat their data less; with dropout, the optimum is unchanged at 6M tokens and moves to the smallest model only at 1.5M, where m1 with dropout gives the best result we observed. Whether this survives a FLOP-matched budget or harder data remains open.